

Reviewer Pack — Euler Characteristic of Clique Complex Bounds Tseitin Resolu...

Entry 7ce08e926626 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 13:17:57 UTC

Euler Characteristic of Clique Complex Bounds Tseitin Resolution Length

Entry ID: 7ce08e926626 Verdict: INCONCLUSIVE Recorded: 2026-05-20 13:17:57 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology **Field B** (complexity object): Resolution Length of Tseitin Formulas

Statement:

For a graph G with n vertices, the Resolution length of the Tseitin formula on G is at least $2^{\{\Omega(\chi(G))\}}$, where $\chi(G)$ is the Euler characteristic of the clique complex of G .

Rationale (proposer’s reasoning):

The Euler characteristic of the clique complex captures the topological complexity of the graph’s structure. High Euler characteristic may indicate a more intricate topology, which could correlate with the exponential growth in resolution length required to refute Tseitin formulas.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREG (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 23150296b230e0a7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 40
    edges = []
    for _ in range(n * (n - 1) // 2):
        u, v = random.sample(range(n), 2)
        if u < v and (u, v) not in edges and (v, u) not in edges:
            edges.append((u, v))

    def clique_complex(G):
        cliques = []
        for r in range(1, n + 1):
            for subset in itertools.combinations(range(n), r):
                is_clique = True
                for i in range(r):
                    for j in range(i + 1, r):
                        if (subset[i], subset[j]) not in G:
                            is_clique = False
                            break
                    if not is_clique:
                        break
                if is_clique:
                    cliques.append(subset)
        return cliques

    def euler_characteristic(cliques):
        V = n
        E = len(edges)
        F = len(cliques)
        C = 0
        for clique in cliques:
            C += (-1) ** (len(clique) - 2) * math.comb(len(clique), 2)
        return V - E + F + C

    def resolution_length(G):
        clauses = []
```

```

variables = set()
for u, v in edges:
    var_u = f"x{u}"
    var_v = f"x{v}"
    variables.add(var_u)
    variables.add(var_v)
    clauses.append([var_u, var_v])
    clauses.append([-var_u, -var_v])
    clauses.append([var_u, -var_v])
    clauses.append([-var_u, var_v])

def unit_propagate(clauses):
    while True:
        changed = False
        for clause in clauses:
            if len(clause) == 1:
                literal = clause[0]
                if literal > 0:
                    new_clauses = [c for c in clauses if literal not in c and -literal not in c]
                    clauses = new_clauses
                    changed = True
                else:
                    return False
        if not changed:
            break
    return True

def dpll(clauses, assignment):
    unit_propagate(clauses)
    if not clauses:
        return True
    literal = next((l for l in variables if l not in assignment and -l not in assignment), None)
    if literal is None:
        return False
    assignment[literal] = True
    new_clauses = [c for c in clauses if literal not in c and -literal not in c]
    if dpll(new_clauses, assignment):
        return True
    del assignment[literal]
    assignment[-literal] = True
    new_clauses = [c for c in clauses if -literal not in c and literal not in c]
    if dpll(new_clauses, assignment):
        return True
    del assignment[-literal]
    return False

return len(clauses)

cliques = clique_complex(edges)
chi_G = euler_characteristic(cliques)
res_length = resolution_length(edges)

metric_name = "Resolution length"
metric_value = res_length
instances_tested = 1
conjecture_holds = res_length >= 2 ** (0.5 * chi_G)

```

```

counterexample = "" if conjecture_holds else f"Graph with n={n}, chi(G)={chi_G}, res_length={res_length}"

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph with n=40, chi(G)={chi_G}, res_length={res_length}\"")
    else:
        print("RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=30")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing data; cannot assess support fraction or counterexamples.
 | next: Run test with extended timeout or optimized parameters to complete evaluation

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	136236
2	propose	ollama_remote	qwen3:8b	0	0	92665
3	preregistration	ollama_remote	qwen3:8b	0	0	31759
4	novelty	ollama_remote	qwen3:8b	0	0	27349
5	novelty	ollama_remote	qwen3:8b	0	0	17239
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14038
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11846
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10613
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13127
10	judge	ollama_remote	qwen3:8b	0	0	90708

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 445580 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/7ce08e926626.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7ce08e926626.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7ce08e926626.tar.gz` (if generated)